

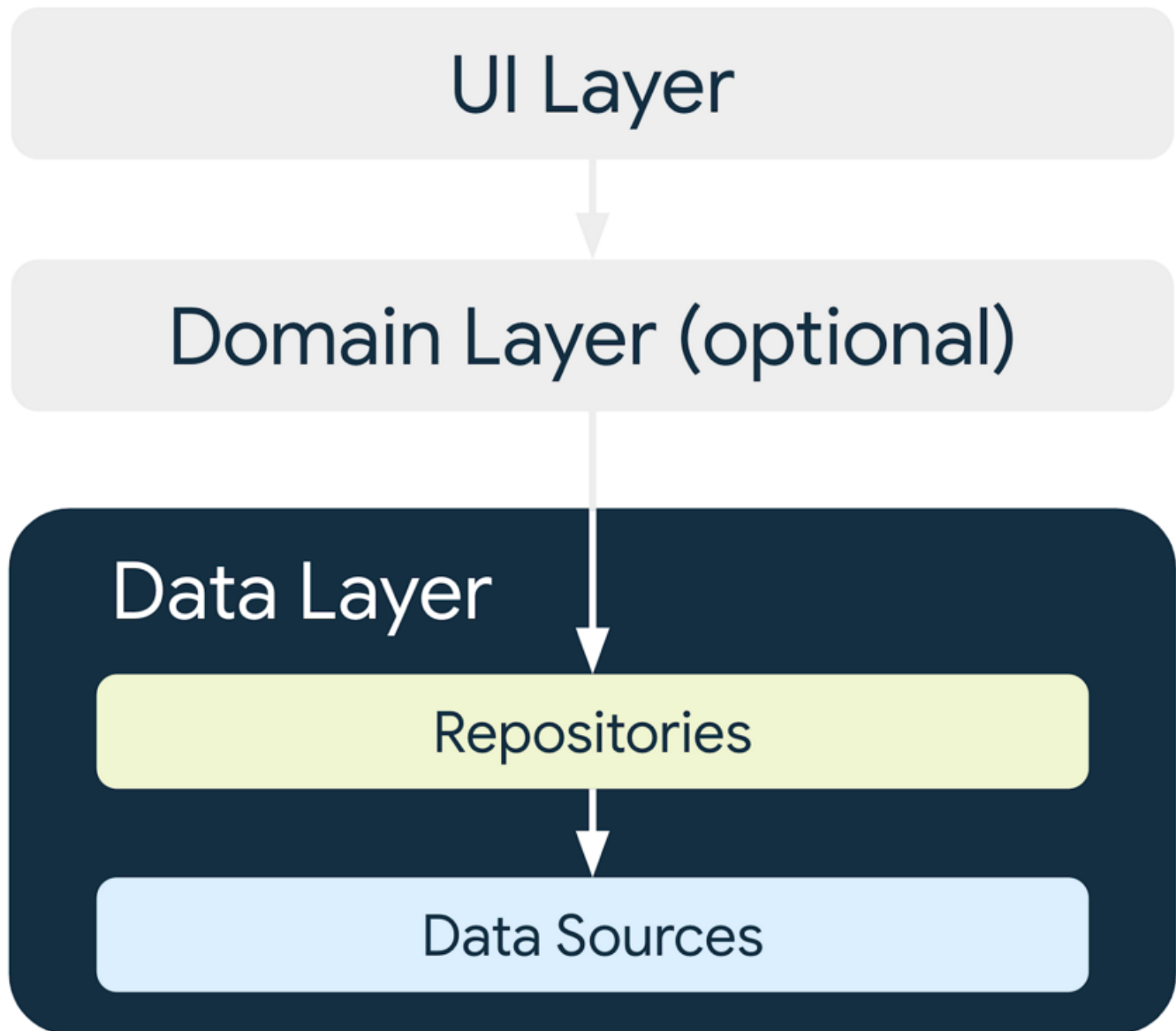
Практическая работа 31. Сохранение данных с помощью Room

В большинстве приложений производственного качества есть данные, которые необходимо сохранять. Например, приложение может хранить плейлист песен, пункты списка дел, записи о расходах и доходах, каталог созвездий или историю личных данных. В таких случаях для хранения постоянных данных используется база данных.

Room - это библиотека персистентности, входящая в состав Android Jetpack. Room - это слой абстракции поверх базы данных SQLite. SQLite использует специализированный язык (SQL) для выполнения операций с базой данных. Вместо того чтобы использовать SQLite напрямую, Room упрощает работу по настройке, конфигурированию и взаимодействию базы данных с приложением. Room также обеспечивает проверку операторов SQLite во время компиляции.

Уровень абстракции - это набор функций, которые скрывают базовую реализацию/сложность. Он предоставляет интерфейс к существующему набору функций, как в данном случае SQLite.

На рисунке ниже показано, как Room в качестве источника данных вписывается в общую архитектуру, рекомендуемую в этом курсе. Room - источник данных.



Необходимые условия

- Умение создавать базовый пользовательский интерфейс (UI) для приложения Android с помощью Jetpack Compose.
- Умение использовать такие составные элементы, как Text, Icon, IconButton и LazyColumn.
- Умение использовать композит NavHost для определения маршрутов и экранов в приложении.
- Возможность навигации между экранами с помощью NavController.
- Знакомство с компонентом ViewModel архитектуры Android. Умение использовать ViewModelProvider.Factory для инстанцирования ViewModel.
- Знакомство с основами параллелизма.
- Умение использовать корутины для выполнения длительных задач.
- Базовые знания баз данных SQLite и языка SQL.

Что вы узнаете

- Как создавать и взаимодействовать с базой данных SQLite с помощью библиотеки Room.
- Как создать сущность, объект доступа к данным (DAO) и классы базы данных.
- Как использовать DAO для сопоставления функций Kotlin с SQL-запросами.

Что вы будете создавать

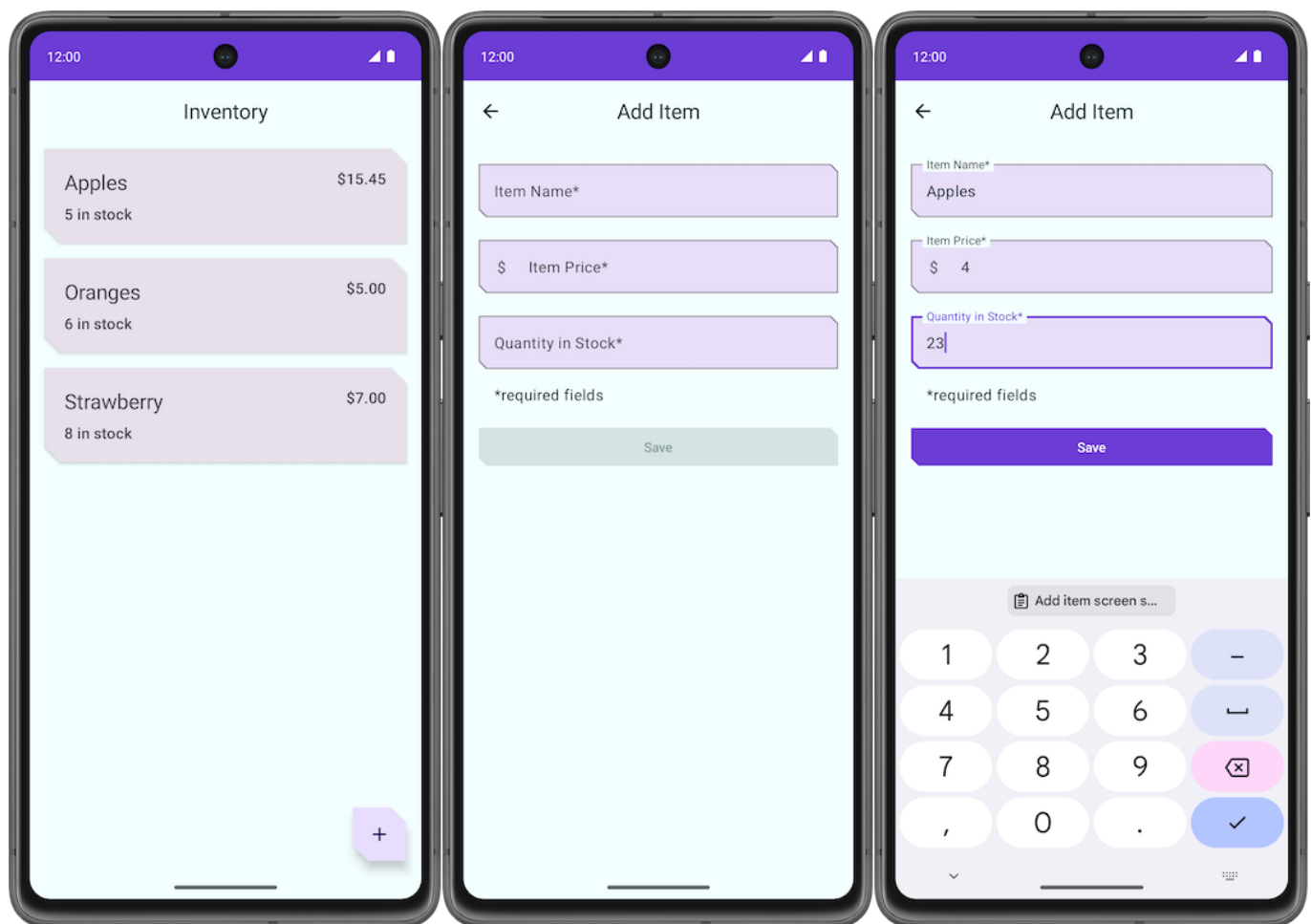
- Вы создадите приложение Inventory, которое сохраняет инвентарные объекты в базу данных SQLite.

Что вам понадобится

- Стартовый код для приложения «Инвентаризация»
- Компьютер с Android Studio
- Устройство или эмулятор с уровнем API 26 или выше.

Обзор приложения

В этом уроке вы работаете со стартовым кодом приложения Inventory и добавляете в него слой базы данных с помощью библиотеки Room. Финальная версия приложения отображает список предметов из базы данных инвентаря. У пользователя есть возможность добавить новый предмет, обновить существующий и удалить предмет из базы данных инвентаризации. В этом уроке вы сохраняете данные о предметах в базе данных Room. Остальные функции приложения будут реализованы в следующем уроке.



Примечание: приведенные выше скриншоты относятся к финальной версии приложения в конце пути, а не к концу этого урока. Эти скриншоты дают представление о финальной версии приложения.

3. Обзор стартового приложения

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git
$ cd basic-android-kotlin-compose-training-inventory-app
$ git checkout starterv
```

Вы можете просмотреть код в GitHub-репозитории приложения Inventory.

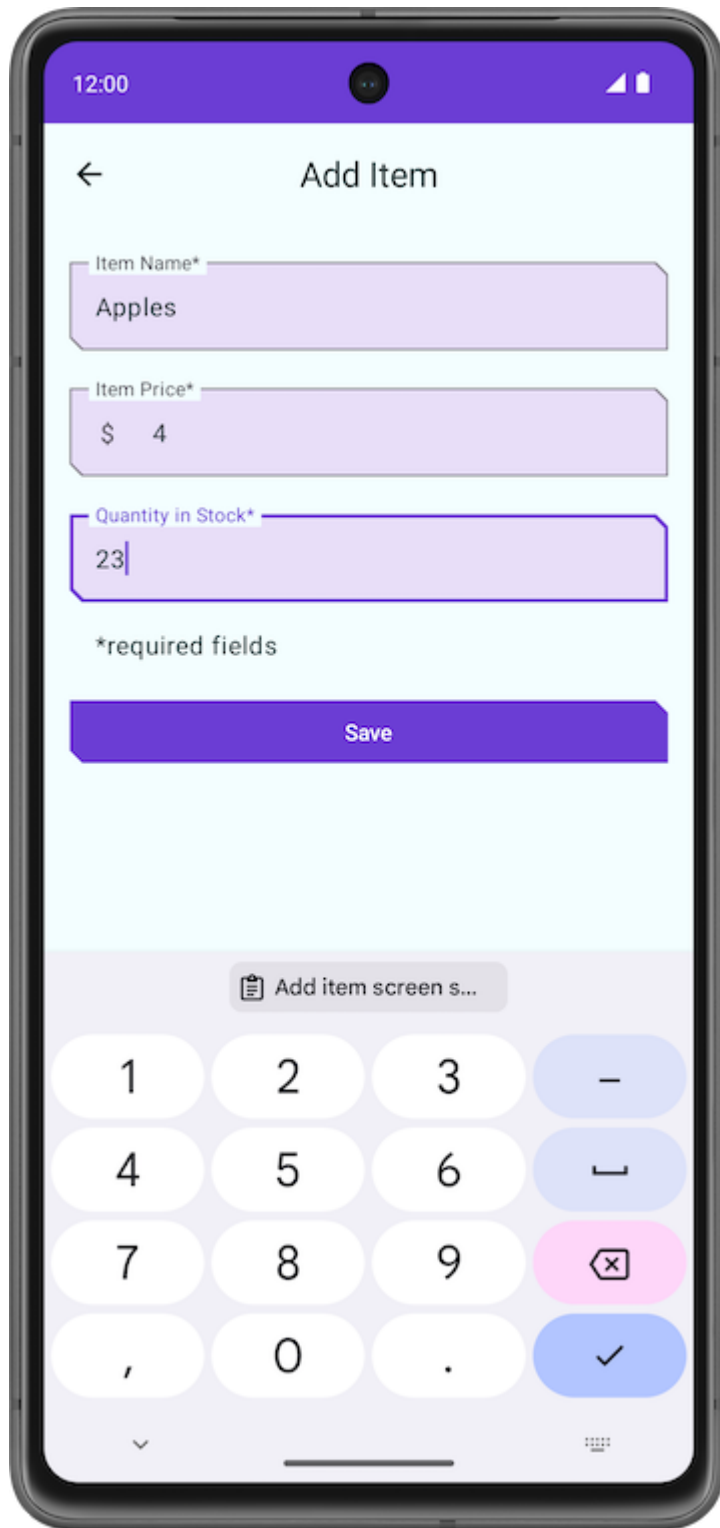
Обзор стартового кода Откройте проект со стартовым кодом в Android Studio. Запустите приложение на Android-устройстве или эмуляторе. Убедитесь, что эмулятор или подключенное устройство работает с API уровня 26 или выше. Инспектор базы данных работает на эмуляторах/устройствах с уровнем API 26 и выше.

Примечание: Инспектор баз данных позволяет просматривать, запрашивать и изменять базы данных вашего приложения во время его работы. Инспектор баз данных работает с обычным SQLite или с библиотеками, построенными на основе SQLite, такими как Room.

Обратите внимание, что в приложении нет данных об инвентаре. Нажмите плавающую кнопку действий (FAB), которая позволяет добавлять новые предметы в базу данных. The app navigates to a new screen where you can enter details for the new item.



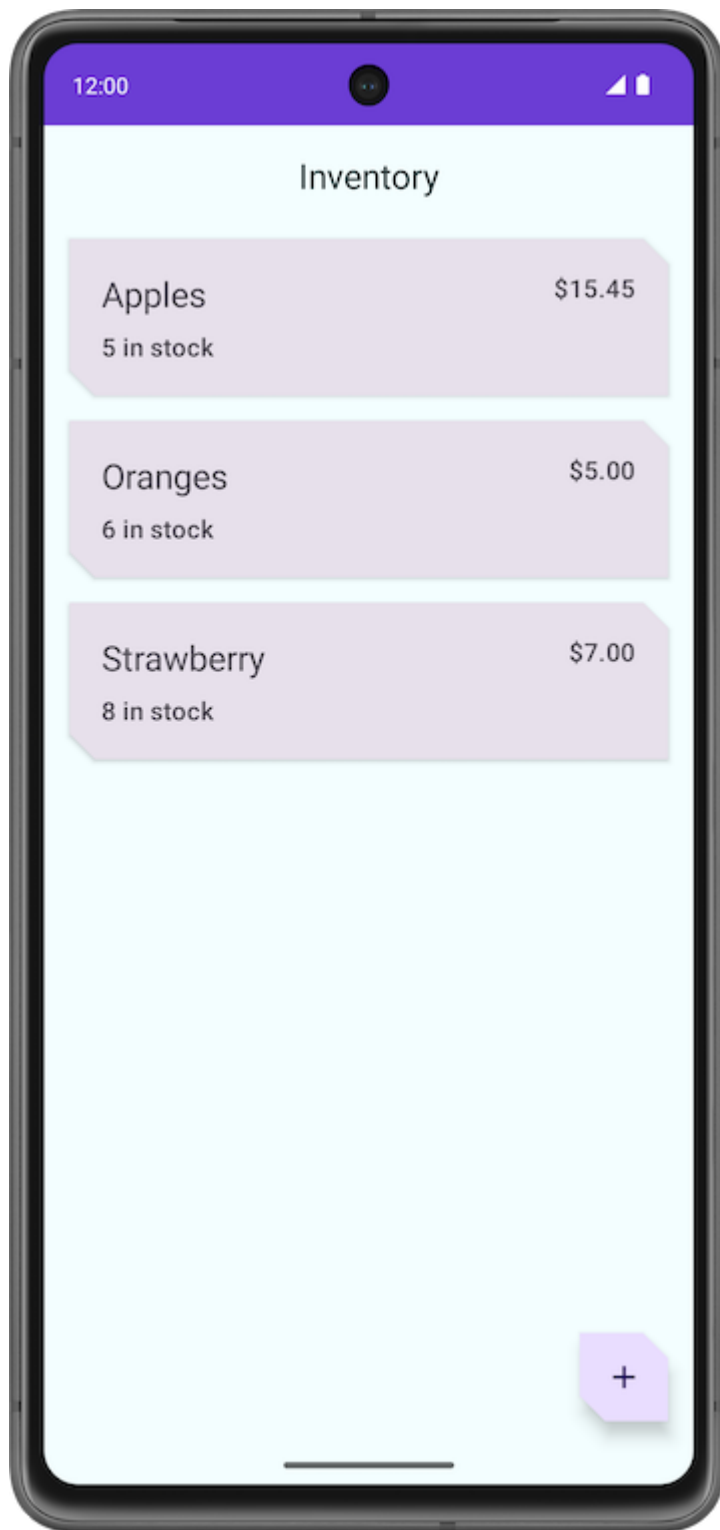
Problems with the starter code In the Add Item screen, enter an item's details like name, price, and quantity of the Item. Tap Save. The Add Item screen is not closed, but you can navigate back using the back key. The save functionality is not implemented, so the item details are not saved. Notice that the app is incomplete and the Save button functionality is not implemented.



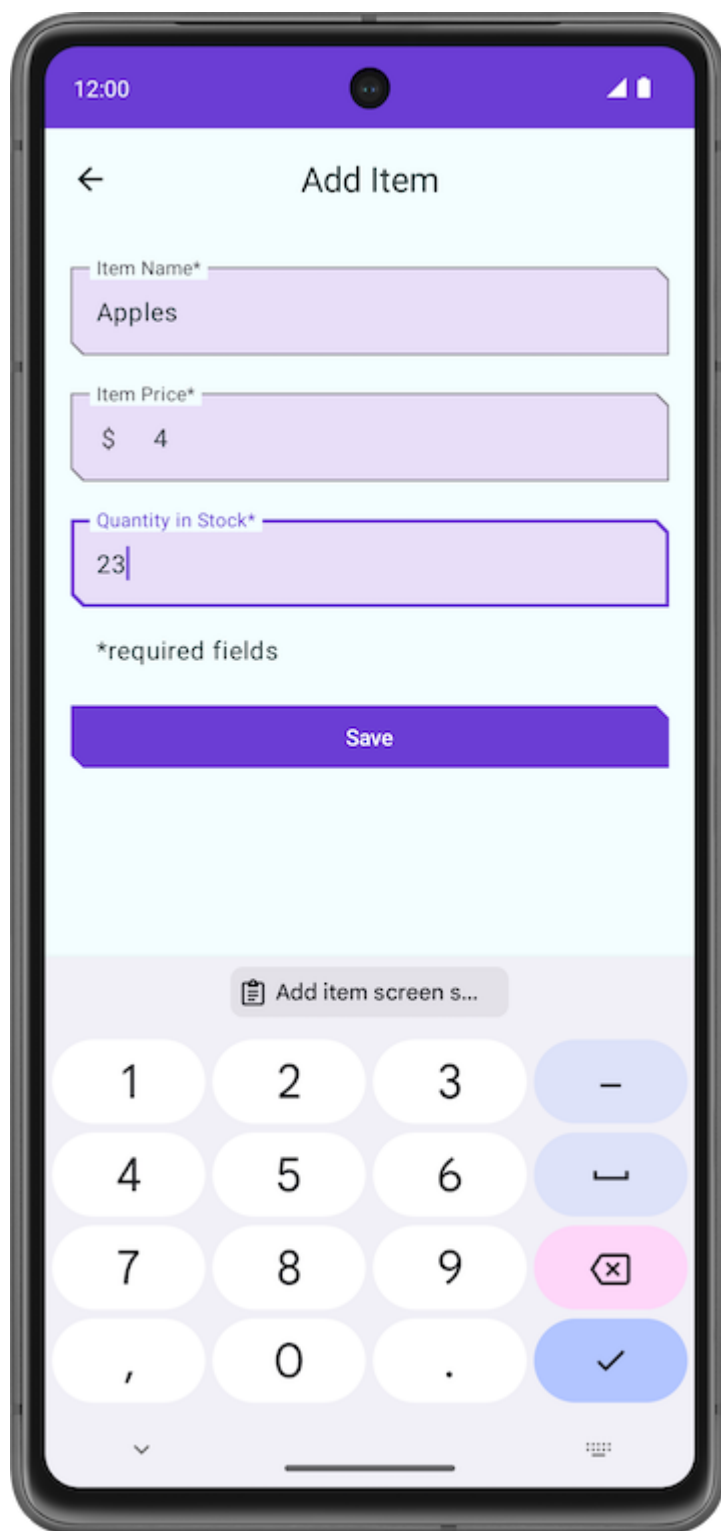
В этом кодовом примере вы добавляете код, который использует Room для сохранения данных инвентаризации в базе данных SQLite. Для взаимодействия с базой данных SQLite вы используете библиотеку постоянства Room.

Описание кода Стартовый код, который вы загрузили, содержит заранее разработанные макеты экранов. На этом пути вы сосредоточитесь на реализации логики базы данных. В следующем разделе мы кратко рассмотрим некоторые файлы, чтобы вы могли начать работу.

ui/home/HomeScreen.kt Этот файл представляет собой домашний экран, или первый экран приложения, который содержит составные элементы для отображения списка инвентаря. На нем есть FAB + для добавления новых предметов в список. Позже вы отобразите предметы в списке на пути.



ui/item/ItemEntryScreen.kt Этот экран похож на ItemEditScreen.kt. Оба они содержат текстовые поля для подробной информации об элементе. Этот экран отображается при нажатии на FAB на главном экране. ItemEntryViewModel.kt является соответствующей ViewModel для этого экрана.



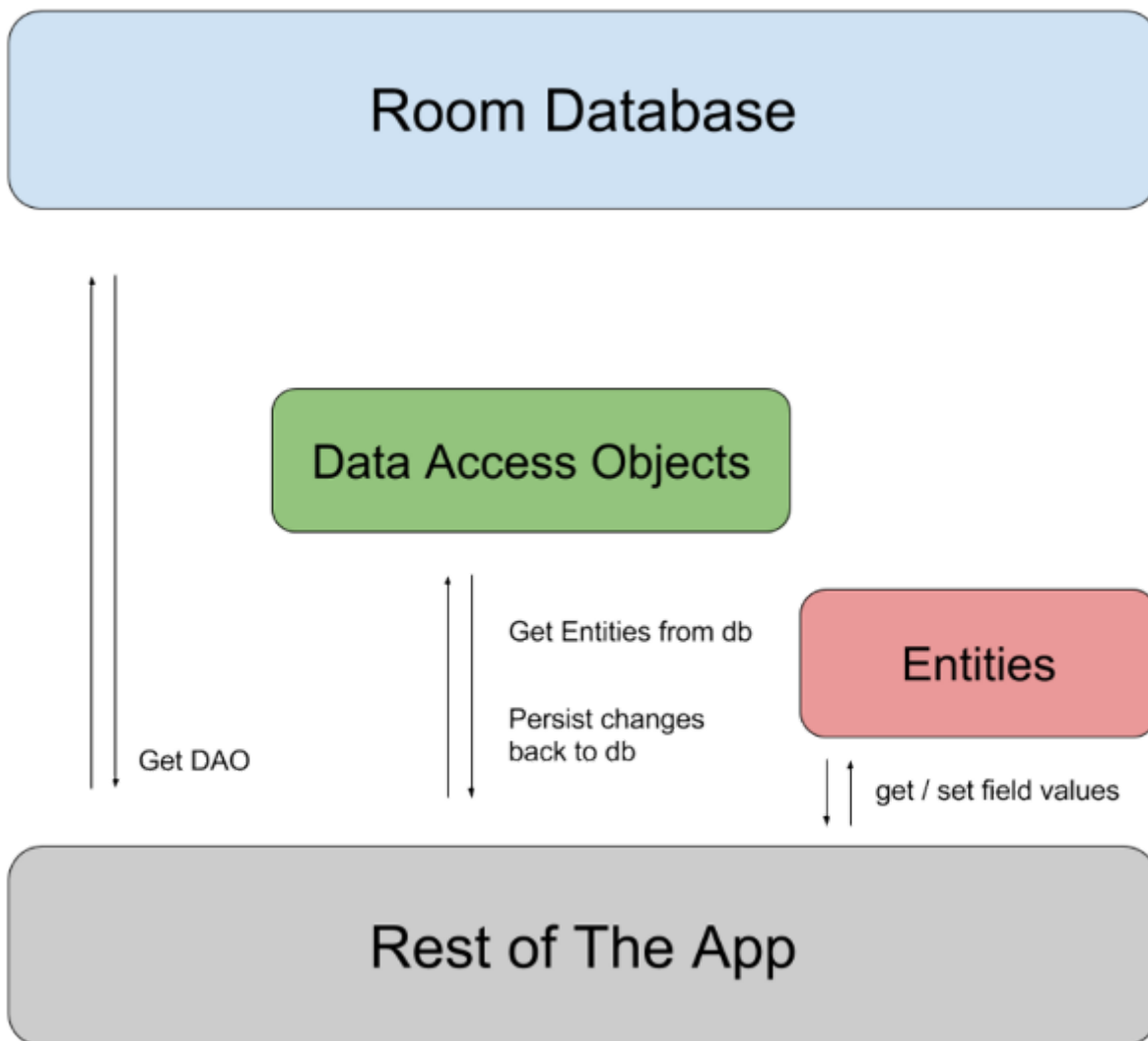
ui/navigation/InventoryNavGraph.kt Этот файл представляет собой навигационный граф для всего приложения.

4. Основные компоненты Room

Kotlin предоставляет простой способ работы с данными через классы данных. Хотя работать с данными в памяти с помощью классов данных очень просто, когда речь заходит о сохранении данных, необходимо преобразовать их в формат, совместимый с хранением в базе данных. Для этого вам понадобятся таблицы для хранения данных и запросы для доступа к ним и их изменения.

Следующие три компонента Room делают эти рабочие процессы беспрепятственными.

Сущности Room представляют собой таблицы в базе данных вашего приложения. Вы используете их для обновления данных, хранящихся в строках таблиц, и для создания новых строк для вставки. ДАО Room предоставляют методы, которые ваше приложение использует для получения, обновления, вставки и удаления данных в базе данных. Класс базы данных Room Database - это класс базы данных, который предоставляет вашему приложению экземпляры ДАО, связанных с этой базой данных. Вы реализуете и узнаете больше об этих компонентах позже в codelab. На следующей диаграмме показано, как компоненты Room взаимодействуют с базой данных.



Добавление зависимостей Room В этой задаче вы добавите необходимые библиотеки компонентов Room в ваши файлы Gradle.

Откройте gradle-файл build.gradle.kts на уровне модуля (Модуль: InventoryApp.app). В блоке зависимостей добавьте зависимости для библиотеки Room, показанные в следующем коде.

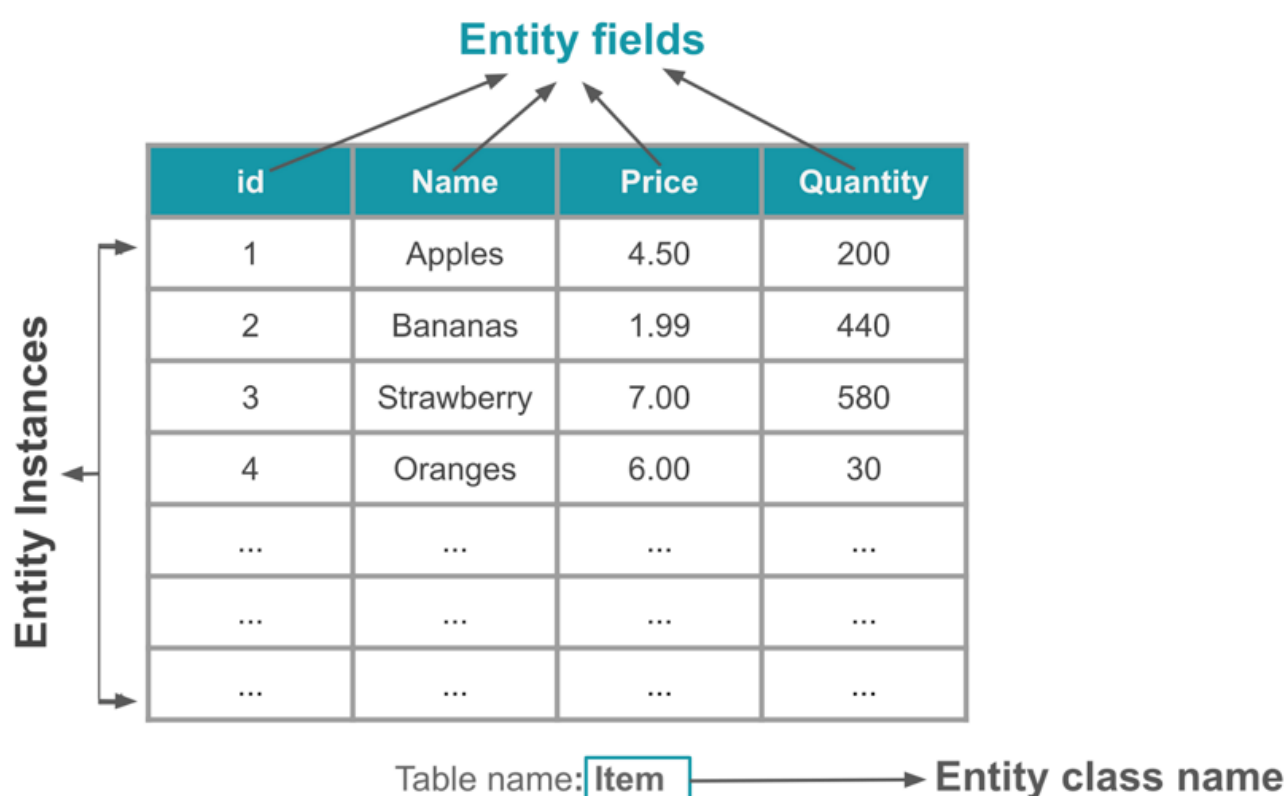
```
//Room
implementation(«androidx.room:room-runtime:${rootProject.extra[»room_version«]}»)
ksp(«androidx.room:room-compiler:${rootProject.extra[»room_version«]}»)
implementation(«androidx.room:room-ktx:${rootProject.extra[»room_version«]}»)
```


KSP - это мощный и в то же время простой API для разбора аннотаций Kotlin.

Примечание: Для зависимостей библиотек в вашем Gradle-файле всегда используйте самые актуальные номера версий стабильных релизов со страницы релизов AndroidX.

5. Создание элемента Entity

Класс Entity определяет таблицу, и каждый экземпляр этого класса представляет собой строку в таблице базы данных. Класс сущности имеет сопоставления, указывающие Room, как он собирается представлять информацию в базе данных и взаимодействовать с ней. В вашем приложении сущность содержит информацию о товарных позициях, такую как название позиции, цена позиции и количество доступных позиций.



Аннотация @Entity отмечает класс как класс сущности базы данных. Для каждого класса Entity приложение создает таблицу базы данных для хранения элементов. Каждое поле сущности представляется как столбец в базе данных, если не указано иное (подробнее см. документацию по сущностям). Каждый экземпляр сущности, хранящийся в базе данных, должен иметь первичный ключ. Первичный ключ используется для уникальной идентификации каждой записи/входа в таблицах вашей базы данных. После того как приложение назначает первичный ключ, его нельзя изменить; он представляет объект сущности до тех пор, пока он существует в базе данных.

В этом задании вы создадите класс Entity и определите поля для хранения следующей инвентарной информации для каждого товара: Int для хранения первичного ключа, String для хранения названия товара, double для хранения цены товара и Int для хранения количества товара на складе.

Откройте начальный код в Android Studio. Откройте пакет данных в базовом пакете com.example.inventory. Внутри пакета данных откройте Kotlin-класс Item, который представляет

сущность базы данных в вашем приложении.

```
// Не нужно копировать, это часть стартового кода
class Item(
    val id: Int,
    val name: String,
    val цена: Double,
    val количество: Int
)
```

Примечание: напомним, что первичный конструктор является частью заголовка класса в классе Kotlin. Он идет после имени класса (и необязательных параметров типа).

Классы данных Классы данных в основном используются для хранения данных в Kotlin. Они определяются с помощью ключевого слова `data`. Объекты классов данных в Kotlin имеют некоторые дополнительные преимущества. Например, компилятор автоматически генерирует утилиты для сравнения, печати и копирования, такие как `toString()`, `copy()` и `equals()`.

Пример: ``KT" // Пример класса данных с 2 свойствами. `data class User(val firstName: String, val lastName: String){ }`

Для обеспечения согласованности и осмысленного поведения сгенерированного кода классы данных должны удовлетворять следующим требованиям:

Первичный конструктор должен иметь хотя бы один параметр.

Все параметры первичного конструктора должны быть `val` или `var`.

Классы данных не могут быть абстрактными, открытыми или запечатанными.

> Предупреждение: Компилятор использует только свойства, определенные внутри первичного конструктора, для автоматически генерируемых функций. Компилятор исключает свойства, объявленные в теле класса, из генерируемых реализаций.

Чтобы узнать больше о классах `Data`, ознакомьтесь с документацией по классам `Data`.

Чтобы преобразовать определение класса `Item` в класс данных, добавьте к нему ключевое слово `data`.

```
```kt
data class Item(
 val id: Int,
 val name: String,
 val цена: Double,
 val количество: Int
)
```

Над объявлением класса `Item` аннотируйте класс данных с помощью `@Entity`. Используйте аргумент `tableName`, чтобы задать элементам имя таблицы SQLite.

```
import androidx.room.Entity

@Entity(tableName = "items")
data class Item(
 ...
)
```

Примечание: Аннотация `@Entity` имеет несколько возможных аргументов. По умолчанию (без аргументов для `@Entity`) имя таблицы совпадает с именем класса. Используйте аргумент `tableName`, чтобы настроить имя таблицы. Для простоты вы используете элемент. Существует несколько других аргументов для `@Entity`, которые вы можете найти в документации по Entity.

Аннотируйте свойство `id` аргументом `@PrimaryKey`, чтобы сделать `id` первичным ключом. Первичный ключ - это идентификатор, позволяющий однозначно идентифицировать каждую запись/вход в таблицу `Item`

```
import androidx.room.PrimaryKey

@Entity(tableName = «items»)
data class Item(
 @PrimaryKey
 val id: Int,
 ...
)
```

Присвойте `id` значение по умолчанию 0, которое необходимо для автоматической генерации значений `id`. Добавьте параметр `autoGenerate` в аннотацию `@PrimaryKey`, чтобы указать, должен ли столбец первичного ключа генерироваться автоматически. Если `autoGenerate` имеет значение `true`, Room будет автоматически генерировать уникальное значение для столбца первичного ключа при вставке нового экземпляра сущности в базу данных. Это гарантирует, что каждый экземпляр сущности будет иметь уникальный идентификатор, без необходимости вручную присваивать значения столбцу первичного ключа

```
data class Item(
 @PrimaryKey(autoGenerate = true)
 val id: Int = 0,
 // ...
)
```

Отлично! Теперь, когда вы создали класс Entity, вы можете создать объект доступа к данным (DAO) для доступа к базе данных.

## 6. Создание элемента DAO

Объект доступа к данным (DAO) - это паттерн, который можно использовать для отделения уровня персистентности от остальной части приложения путем предоставления абстрактного интерфейса. Такая изоляция соответствует принципу единой ответственности, который вы уже видели в предыдущих уроках.

Функциональность DAO заключается в том, чтобы скрыть все сложности, связанные с выполнением операций с базой данных, в нижележащем слое персистентности, отдельно от остальной части приложения. Это позволяет изменять слой данных независимо от кода, который использует эти данные.



В этой задаче вы определяете DAO для Room. DAO - это основные компоненты Room, которые отвечают за определение интерфейса доступа к базе данных.

Созданный вами DAO - это пользовательский интерфейс, предоставляющий удобные методы для запроса/получения, вставки, удаления и обновления базы данных. Room генерирует реализацию этого класса во время компиляции.

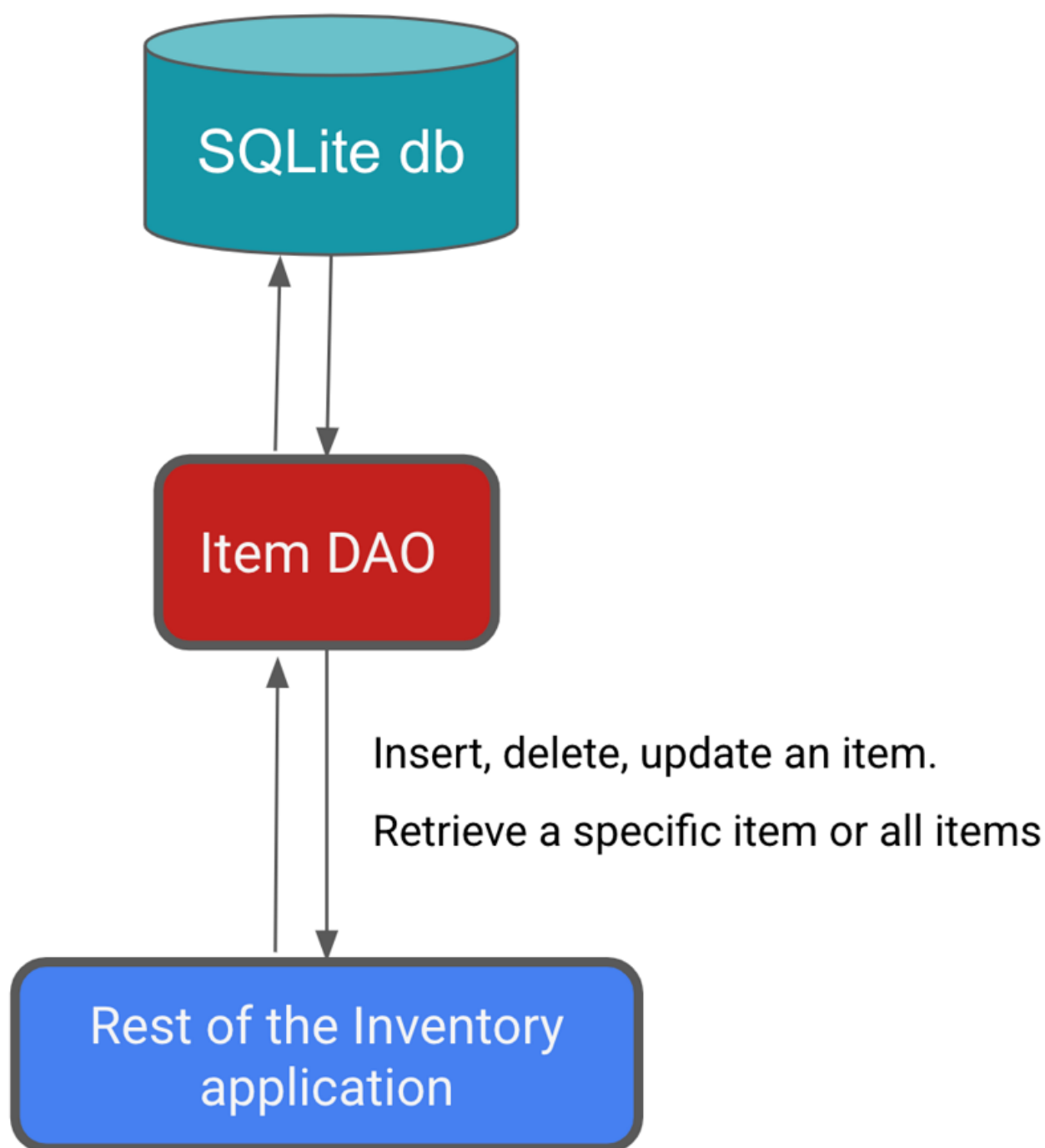
Библиотека Room предоставляет удобные аннотации, такие как `@Insert`, `@Delete` и `@Update`, для определения методов, которые выполняют простые вставки, удаления и обновления, не требуя написания SQL-запросов.

Если вам нужно определить более сложные операции вставки, удаления, обновления или запросить данные в базе данных, используйте аннотацию `@Query`.

В качестве дополнительного бонуса, по мере написания запросов в Android Studio компилятор проверяет SQL-запросы на наличие синтаксических ошибок.

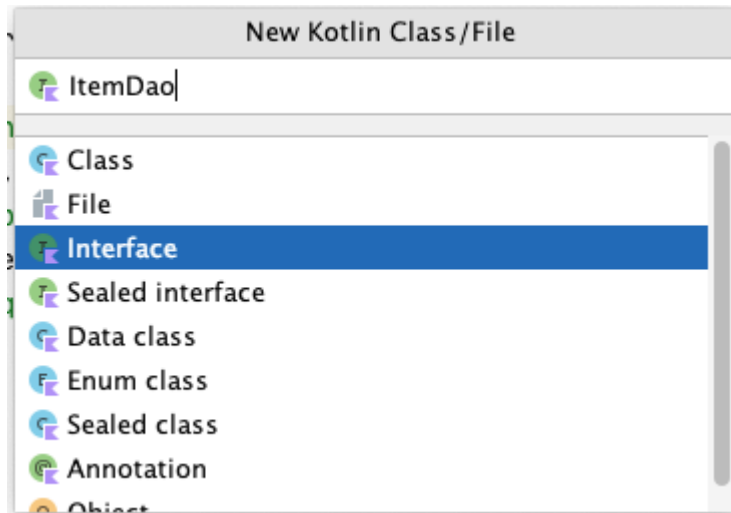
Для приложения «Инвентаризация» вам нужна возможность делать следующее:

Вставить или добавить новый элемент. Обновлять существующий товар, чтобы изменить его название, цену и количество. Получить конкретный элемент на основе его первичного ключа `id`. Получить все товары, чтобы отобразить их. Удалить запись в базе данных.



Выполните следующие шаги, чтобы реализовать DAO элементов в вашем приложении:

В пакете данных создайте Kotlin-интерфейс `ItemDao.kt`.



Аннотируйте интерфейс ItemDao с помощью @Dao.

```
import androidx.room.Dao

@Dao
interface ItemDao {
}
```

Внутри тела интерфейса добавьте аннотацию @Insert. Под @Insert добавьте функцию insert(), которая принимает в качестве аргумента экземпляр элемента класса Entity. Пометьте функцию ключевым словом suspend, чтобы она выполнялась в отдельном потоке. Выполнение операций с базой данных может занять много времени, поэтому они должны выполняться в отдельном потоке. Room не позволяет обращаться к базе данных в главном потоке.

```
import androidx.room.Insert

@Insert
suspend fun insert(item: Item)
```

При вставке элементов в базу данных могут возникать конфликты. Например, в нескольких местах кода пытаются обновить сущность с разными, конфликтующими значениями, например, с одним и тем же первичным ключом. Сущность - это строка в БД. В приложении Inventory мы вставляем сущность только в одном месте - на экране Add Item, поэтому мы не ожидаем никаких конфликтов и можем установить стратегию конфликтов на Ignore.

Добавьте аргумент onConflict и присвойте ему значение OnConflictStrategy.IGNORE. Аргумент onConflict указывает комнате, что делать в случае возникновения конфликта. Стратегия OnConflictStrategy.IGNORE игнорирует новый элемент.

Чтобы узнать больше о доступных стратегиях конфликта, ознакомьтесь с документацией по OnConflictStrategy.

```
import androidx.room.OnConflictStrategy

@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insert(item: Item)
```

Теперь Room генерирует весь необходимый код для вставки элемента в базу данных. Когда вы вызываете любую из функций DAO, помеченных аннотациями Room, Room выполняет соответствующий SQL-запрос к базе данных. Например, когда вы вызываете описанный выше метод insert() из вашего кода на Kotlin, Room выполняет SQL-запрос для вставки сущности в базу данных.

Добавьте новую функцию с аннотацией @Update, которая принимает Item в качестве параметра. Обновляемая сущность будет иметь тот же первичный ключ, что и переданная сущность. Вы можете обновить некоторые или все другие свойства сущности.

Подобно методу insert(), пометьте эту функцию ключевым словом suspend.

```
import androidx.room.Update

@Update
suspend fun update(item: Item)
```

Добавьте еще одну функцию с аннотацией @Delete для удаления элемента(ов) и сделайте ее приостанавливающей.

Примечание: Аннотация @Delete удаляет элемент или список элементов. Вам нужно передать сущности, которые вы хотите удалить. Если у вас нет сущности, вам, возможно, придется получить ее перед вызовом функции delete().

```
import androidx.room.Delete

@Delete
suspend fun delete(item: Item)
```

Для остальной функциональности нет удобной аннотации, поэтому вам придется использовать аннотацию @Query и предоставлять SQLite-запросы.

Напишите SQLite-запрос для получения определенного элемента из таблицы item на основе заданного id. В следующем коде приведен пример запроса, который выбирает все столбцы из таблицы items, где id совпадает с определенным значением, а id является уникальным идентификатором. Пример:

```
// Пример, копировать не нужно
SELECT * from items WHERE id = 1
```

Добавьте аннотацию `@Query`. Используйте SQLite-запрос из предыдущего шага в качестве строкового параметра аннотации `@Query`. Добавьте к `@Query` параметр `String`, который представляет собой SQLite-запрос для получения элемента из таблицы `item`. Теперь в запросе говорится, что нужно выбрать все столбцы из элементов, где `id` совпадает с аргументом `:id`. Обратите внимание, что в запросе для ссылки на аргументы в функции в `:id` используется обозначение двоеточия.

```
@Query(«SELECT * from items WHERE id = :id»)
```

После аннотации `@Query` добавьте функцию `getItem()`, которая принимает аргумент `Int` и возвращает `Flow`.

```
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Query(«SELECT * from items WHERE id = :id»)
fun getItem(id: Int): Flow<Item>
```

Рекомендуется использовать `Flow` в слое персистентности. Используя `Flow` в качестве возвращаемого типа, вы получаете уведомления при каждом изменении данных в базе данных. Room обновляет этот `Flow` за вас, что означает, что вам нужно только один раз явно получить данные. Такая настройка полезна для обновления списка инвентаря, которое вы реализуете в следующем уроке. Благодаря типу возврата `Flow`, Room также выполняет запрос в фоновом потоке. Вам не нужно явно делать эту функцию приостановленной и вызывать ее в области действия корутины.

Примечание: `Flow` в базе данных Room может поддерживать данные в актуальном состоянии, выдавая уведомление при каждом изменении данных в базе. Это позволяет наблюдать за данными и соответствующим образом обновлять пользовательский интерфейс.

Добавьте `@Query` с функцией `getAllItems()`. Пусть SQLite-запрос возвращает все столбцы из таблицы элементов, упорядоченные по возрастанию. Пусть функция `getAllItems()` возвращает список сущностей `Item` в виде потока `Flow`. Room будет обновлять этот поток за вас, что означает, что вам нужно явно получить данные только один раз.

```
@Query("SELECT * from items ORDER BY name ASC")
fun getAllItems(): Flow<List<Item>>
```

Завершено `ItemDao`:

```
import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
```



```
import kotlinx.coroutines.flow.Flow

@Dao
interface ItemDao {
 @Insert(onConflict = OnConflictStrategy.IGNORE)
 suspend fun insert(item: Item)

 @Update
 suspend fun update(item: Item)

 @Delete
 suspend fun delete(item: Item)

 @Query("SELECT * from items WHERE id = :id")
 fun getItem(id: Int): Flow<Item>

 @Query("SELECT * from items ORDER BY name ASC")
 fun getAllItems(): Flow<List<Item>>
}
```

Хотя вы не увидите никаких видимых изменений, соберите приложение, чтобы убедиться, что в нем нет ошибок.

## 7. Создание экземпляра базы данных

---

В этой задаче вы создаете базу данных RoomDatabase, которая использует ваши сущности и DAO из предыдущих задач. Класс базы данных определяет список сущностей и DAO.

Класс базы данных предоставляет вашему приложению экземпляры определенных вами DAO. В свою очередь, приложение может использовать DAO для получения данных из базы данных в виде экземпляров связанных с ними объектов сущностей. Приложение также может использовать определенные объекты данных для обновления строк из соответствующих таблиц или для создания новых строк для вставки.

Вам необходимо создать абстрактный класс RoomDatabase и аннотировать его с помощью @Database. Этот класс имеет один метод, который возвращает существующий экземпляр RoomDatabase, если база данных не существует.

Вот общий процесс получения экземпляра RoomDatabase:

Создайте публичный абстрактный класс, который расширяет RoomDatabase. Новый абстрактный класс, который вы определяете, действует как держатель базы данных. Класс, который вы определяете, является абстрактным, потому что Room создает его реализацию за вас. Аннотируйте класс с помощью @Database. В аргументах перечислите сущности для базы данных и задайте номер версии. Определите абстрактный метод или свойство, возвращающее экземпляр ItemDao, и Room создаст его реализацию за вас. Вам нужен только один экземпляр базы данных RoomDatabase для всего приложения, поэтому сделайте базу данных RoomDatabase синглтоном. Используйте Room's Room.databaseBuilder для создания базы данных (item\_database), только если она не существует. В противном случае верните существующую базу данных. Создание базы данных В пакете данных создайте Kotlin-класс

InventoryDatabase.kt. В файле InventoryDatabase.kt сделайте класс InventoryDatabase абстрактным классом, расширяющим RoomDatabase. Аннотируйте класс с помощью @Database. Не обращайте внимания на ошибку отсутствия параметров, которую вы исправите на следующем шаге.

```
import androidx.room.Database
import androidx.room.RoomDatabase

@Database
абстрактный класс InventoryDatabase : RoomDatabase() {}
```

Аннотация @Database требует несколько аргументов, чтобы Room мог создать базу данных.

Укажите Item как единственный класс со списком сущностей. Установите версию 1. Каждый раз, когда вы меняете схему таблицы базы данных, вы должны увеличивать номер версии. Установите exportSchema в false, чтобы не хранить резервные копии истории версий схемы.

```
@Database(entities = [Item::class], version = 1, exportSchema = false)
```

Внутри тела класса объявите абстрактную функцию, возвращающую ItemDao, чтобы база данных знала о DAO.

```
abstract fun itemDao(): ItemDao
```

Ниже абстрактной функции определите объект-компаньон, который позволяет получить доступ к методам создания или получения базы данных и использует имя класса в качестве классификатора.

```
объект-компаньон {}
```

Внутри объекта-компаньона объявите для базы данных частную переменную Instance с возможностью обнуления и инициализируйте ее значением null. Переменная Instance хранит ссылку на базу данных, если она была создана. Это помогает поддерживать единственный экземпляр базы данных, открытый в определенный момент времени, что является дорогостоящим ресурсом для создания и поддержки.

Аннотируйте Instance с помощью @Volatile. Значение переменной volatile никогда не кэшируется, а все операции чтения и записи выполняются в основную память и из нее. Эти особенности позволяют гарантировать, что значение Instance всегда актуально и одинаково для всех потоков выполнения. Это означает, что изменения, внесенные одним потоком в Instance, сразу же видны всем остальным потокам.

```
@Volatile
private var Instance: InventoryDatabase? = null
```

Ниже Instance, находясь внутри объекта-компаньона, определите метод `getDatabase()` с параметром `Context`, который нужен создателю базы данных. Верните тип `InventoryDatabase`. Появляется сообщение об ошибке, поскольку `getDatabase()` пока ничего не возвращает.

```
import android.content.Context

fun getDatabase(context: Context): InventoryDatabase {}
```

Несколько потоков могут запрашивать экземпляр базы данных одновременно, что приводит к созданию двух баз данных вместо одной. Эта проблема известна как состояние гонки. Обертывание кода для получения базы данных внутри синхронизированного блока означает, что только один поток выполнения за один раз может войти в этот блок кода, что гарантирует, что база данных будет инициализирована только один раз. Используйте синхронизированный блок, чтобы избежать состояния гонки.

Внутри `getDatabase()` верните переменную `Instance` - или, если `Instance` равна `null`, инициализируйте ее внутри блока `synchronized{}.` Для этого используйте оператор `elvis()` 😊. Передайте `this`, объект-компаньон. Вы исправите ошибку на последующих шагах.

```
return Instance ?: synchronized(this) { }
```

Внутри блока `synchronized` используйте конструктор базы данных для получения базы данных. Продолжайте игнорировать ошибки, которые вы исправите в следующих шагах.

```
import androidx.room.Room

Room.databaseBuilder()
```

Внутри синхронизированного блока используйте конструктор базы данных для получения базы данных. Передайте контекст приложения, класс базы данных и имя базы данных - `item_database` - в `Room.databaseBuilder()`.

```
Room.databaseBuilder(context, InventoryDatabase::class.java, «item_database»)
```

Android Studio выдает ошибку `Type Mismatch`. Чтобы устранить эту ошибку, необходимо добавить `build()` в следующих шагах.

Добавьте в сборку необходимую стратегию миграции. Используйте `.fallbackToDestructiveMigration()`.

```
.fallbackToDestructiveMigration()
```

Примечание: Обычно вы предоставляете объект миграции со стратегией миграции при изменении схемы. Объект миграции - это объект, определяющий, как вы берете все строки со старой схемой и преобразуете их в строки новой схемы, чтобы не потерять данные. Миграция выходит за рамки этого урока, но этот термин относится к случаям, когда схема меняется и вам нужно перенести дату без потери данных. Поскольку это пример приложения, простая альтернатива - уничтожить и перестроить базу данных, что означает потерю данных инвентаризации. Например, если вы измените что-то в классе сущности, например добавьте новый параметр, вы можете позволить приложению удалить и заново инициализировать базу данных.

Чтобы создать экземпляр базы данных, вызовите `.build()`. Этот вызов устраняет ошибки Android Studio.

```
.build()
```

После `build()` добавьте блок `also` и присвойте ему `Instance =`, чтобы сохранить ссылку на недавно созданный экземпляр `db`.

```
.also { Instance = it }
```

В конце синхронизированного блока верните экземпляр. Ваш финальный код выглядит следующим образом:

```
import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

/**
 * Класс базы данных с синглтоном объекта Instance.
 */
@Database(entities = [Item::class], version = 1, exportSchema = false)
абстрактный класс InventoryDatabase : RoomDatabase() {

 abstract fun itemDao(): ItemDao

 объект-компаньон {
 @Volatile
 private var Instance: InventoryDatabase? = null

 fun getDatabase(context: Context): InventoryDatabase {
 // если Instance не является null, возвращаем его, в противном случае
 // создаем новый экземпляр базы данных.
 return Instance ?: synchronized(this) {
 Room.databaseBuilder(context, InventoryDatabase::class.java,
 «item_database»)
 .build()
 .also { Instance = it }
 }
 }
 }
}
```

```
 }
 }
}
```

Совет: Вы можете использовать этот код в качестве шаблона для своих будущих проектов. Создание экземпляра базы данных RoomDatabase аналогично процессу, описанному в предыдущих шагах. Возможно, вам придется заменить сущности и DAO, характерные для вашего приложения.

Соберите код, чтобы убедиться в отсутствии ошибок.

## 8. Реализация репозитория

В этом задании вы реализуете интерфейс `ItemsRepository` и класс `OfflineItemsRepository` для получения, вставки, удаления и обновления сущностей из базы данных.

Откройте файл `ItemsRepository.kt` в пакете данных. Добавьте в интерфейс следующие функции, которые отображаются на реализацию DAO.

```
import kotlinx.coroutines.flow.Flow

/**
 * Репозиторий, обеспечивающий вставку, обновление, удаление и извлечение [Item] из
 * заданного источника данных.
 */
интерфейс ItemsRepository {
 /**
 * Получение всех элементов из заданного источника данных.
 */
 fun getAllItemsStream(): Flow<List<Item>>

 /**
 * Получение элемента из данного источника данных, который соответствует [id].
 */
 fun getItemStream(id: Int): Flow<Item?>

 /**
 * Вставка элемента в источник данных
 */
 suspend fun insertItem(item: Item)

 /**
 * Удалить элемент из источника данных
 */
 suspend fun deleteItem(item: Item)

 /**
 * Обновление элемента в источнике данных
 */
}
```

```
suspend fun updateItem(item: Item)
}
```

Откройте файл `OfflineItemsRepository.kt` в пакете данных. Передайте в конструктор параметр типа `ItemDao`.

```
class OfflineItemsRepository(private val itemDao: ItemDao) : ItemsRepository
```

В классе `OfflineItemsRepository` переопределите функции, определенные в интерфейсе `ItemsRepository`, и вызовите соответствующие функции из `ItemDao`.

```
import kotlinx.coroutines.flow.Flow

class OfflineItemsRepository(private val itemDao: ItemDao) : ItemsRepository {
 override fun getAllItemsStream(): Flow<List<Item>> = itemDao.getAllItems()

 override fun getItemStream(id: Int): Flow<Item?> = itemDao.getItem(id)

 override suspend fun insertItem(item: Item) = itemDao.insert(item)

 override suspend fun deleteItem(item: Item) = itemDao.delete(item)

 override suspend fun updateItem(item: Item) = itemDao.update(item)
}
```

Реализация класса `AppContainer` В этой задаче вы инстанцируете базу данных и передаете экземпляр DAO классу `OfflineItemsRepository`.

Откройте файл `AppContainer.kt` в пакете данных. Передайте экземпляр `ItemDao()` в конструктор `OfflineItemsRepository`. Инстанцируйте экземпляр базы данных, вызвав `getDatabase()` на классе `InventoryDatabase`, передав контекст, и вызовите `.itemDao()` для создания экземпляра `Dao`.

```
override val itemsRepository: ItemsRepository by lazy {
 OfflineItemsRepository(InventoryDatabase.getDatabase(context).itemDao())
}
```

Теперь у вас есть все строительные блоки для работы с комнатой. Этот код компилируется и запускается, но у вас нет возможности проверить, работает ли он на самом деле. Поэтому сейчас самое время протестировать вашу базу данных. Чтобы завершить тест, вам нужно, чтобы `ViewModel` общалась с базой данных.

## 9. Добавьте функцию сохранения

Вы уже создали базу данных, а классы пользовательского интерфейса были частью начального кода. Чтобы сохранить переходные данные приложения и получить доступ к базе данных, необходимо обновить ViewModel. Ваши ViewModels взаимодействуют с базой данных через DAO и предоставляют данные пользовательскому интерфейсу. Все операции с базой данных должны выполняться вдали от основного потока пользовательского интерфейса; для этого используются корутины и viewModelScope.

Обзор класса состояния пользовательского интерфейса Откройте файл ui/item/ItemEntryViewModel.kt. Класс данных ItemUiState представляет состояние пользовательского интерфейса элемента. Класс данных ItemDetails представляет один элемент.

Стартовый код предоставляет вам три функции расширения:

Функция расширения ItemDetails.toItem() преобразует объект состояния пользовательского интерфейса ItemUiState в тип сущности Item. Функция расширения Item.toItemUiState() преобразует объект сущности Item Room в тип UI-состояния ItemUiState. Функция расширения Item.toItemDetails() преобразует объект сущности Item Room в ItemDetails.

```
// Не нужно копировать, это часть стартового кода
/**
 * Представляет состояние Ui для элемента.
 */
data class ItemUiState(
 val itemDetails: ItemDetails = ItemDetails(),
 val isEntryValid: Boolean = false
)

data class ItemDetails(
 val id: Int = 0,
 val name: String = «»,
 val цена: String = «»,
 val количество: String = «»,
)

/**
 * Функция расширения для преобразования [ItemDetails] в [Item]. Если значение
 [ItemDetails.price]
 * не является действительным [Double], то цена будет установлена в 0.0.
 Аналогично, если значение
 * [ItemDetails.quantity] не является правильным [Int], то количество будет
 установлено в 0.
 */
fun ItemDetails.toItem(): Item = Item(
 id = id,
 name = name,
 price = price.toDoubleOrNull() ?: 0.0,
 quantity = quantity.toIntOrNull() ?: 0
)

fun Item.formatedPrice(): String {
 return NumberFormat.getCurrencyInstance().format(price)
}
```

```

/**
 * Расширение функции для преобразования [Item] в [ItemUiState]
 */
fun Item.toItemUiState(isEntryValid: Boolean = false): ItemUiState = ItemUiState(
 itemDetails = this.toItemDetails(),
 isEntryValid = isEntryValid
)

/**
 * Расширение функции для преобразования [Item] в [ItemDetails]
 */
fun Item.toItemDetails(): ItemDetails = ItemDetails(
 id = id,
 name = name,
 price = price.toString(),
 quantity = quantity.toString()
)

```

Вы используете вышеуказанный класс в моделях представления для чтения и обновления пользовательского интерфейса.

Обновление ViewModel ItemEntry В этой задаче вы передаете репозиторий в файл ItemEntryViewModel.kt. Кроме того, вы сохраняете в базе данных данные об элементе, введенные на экране добавления элемента.

Обратите внимание на частную функцию validateInput() в классе ItemEntryViewModel.

```

// Не нужно копировать, это часть стартового кода
private fun validateInput(uiState: ItemDetails = itemUiState.itemDetails): Boolean
{
 return with(uiState) {
 name.isNotBlank() && price.isNotBlank() && quantity.isNotBlank()
 }
}

```

Приведенная выше функция проверяет, пусты ли имя, цена и количество. Вы используете эту функцию для проверки вводимых пользователем данных перед добавлением или обновлением сущности в базе данных.

Откройте класс ItemEntryViewModel и добавьте частный параметр конструктора по умолчанию типа ItemsRepository.

```

import com.example.inventory.data.ItemsRepository

class ItemEntryViewModel(private val itemsRepository: ItemsRepository) :
 ViewModel() {
}

```



Обновите инициализатор для модели представления ввода элементов в `ui/AppViewModelProvider.kt` и передайте в качестве параметра экземпляр хранилища.

```
object AppViewModelProvider {
 val Factory = viewModelFactory {
 // Другие инициализаторы
 // Инициализатор для ItemEntryViewModel
 инициализатор {
 ItemEntryViewModel(inventoryApplication().container.itemsRepository)
 }
 //...
 }
}
```

Перейдите в файл `ItemEntryViewModel.kt` и в конце класса `ItemEntryViewModel` добавьте функцию `suspend` под названием `saveItem()` для вставки элемента в базу данных Room. Эта функция добавляет данные в базу данных неблокируемым способом.

```
``kt suspend fun saveItem() { }
```

Внутри функции проверьте, является ли `itemUiState` действительным, и преобразуйте его к типу `Item`, чтобы Room мог понять данные. Вызовите `insertItem()` на `itemsRepository` и передайте данные. UI вызывает эту функцию, чтобы добавить данные об элементе в базу данных.

```
``kt
suspend fun saveItem() {
 if (validateInput()) {
 itemsRepository.insertItem(itemUiState.itemDetails.toItem())
 }
}
```

Теперь вы добавили все необходимые функции для добавления сущностей в базу данных. В следующей задаче вы обновите пользовательский интерфейс, чтобы использовать вышеуказанные функции.

Прохождение композиции `ItemEntryBody()` В файле `ui/item/ItemEntryScreen.kt` функция `ItemEntryBody()` `composable` частично реализована для вас как часть кода `stater`. Посмотрите на композит `ItemEntryBody()` в вызове функции `ItemEntryScreen()`.

```
``kt // Не нужно копировать, это часть стартового кода ItemEntryBody(itemUiState =
viewModel.itemUiState, onItemValueChange = viewModel::updateUiState, onSaveClick = { }, модификатор =
Модификатор .padding(innerPadding) .verticalScroll(rememberScrollState()) .fillMaxWidth())
```

Обратите внимание, что состояние пользовательского интерфейса и лямбда `updateUiState` передаются в качестве параметров функции. Посмотрите на определение функции, чтобы увидеть, как обновляется состояние пользовательского интерфейса.

```
```kt
// Не нужно копировать, часть стартового кода
@Composable
fun ItemEntryBody(
    itemUiState: ItemUiState,
    onItemValueChange: (ItemUiState) -> Unit,
    onSaveClick: () -> Unit,
    модификатор: Modifier = Modifier
) {
    Column(
        // ...
    ) {
        ItemInputForm(
            itemDetails = itemUiState.itemDetails,
            onValueChange = onItemValueChange,
            modifier = Modifier.fillMaxWidth()
        )
        Button(
            onClick = onSaveClick,
            enabled = itemUiState.isEntryValid,
            shape = MaterialTheme.shapes.small,
            модификатор = Modifier.fillMaxWidth()
        ) {
            Text(text = stringResource(R.string.save_action))
        }
    }
}
```

В этом композите вы отображаете форму `ItemInputForm` и кнопку `Save`. В составном элементе `ItemInputForm()` отображаются три текстовых поля. Кнопка `Save` активируется только в том случае, если в текстовые поля введен текст. Значение `isEntryValid` равно `true`, если текст во всех текстовых полях правильный (не пустой).



Take a look at the `ItemInputForm()` composable function implementation and notice the `onValueChange` function parameter. You are updating the `itemDetails` value with the value entered by the user in the text fields. By the time the `Save` button is enabled, `itemUiState.itemDetails` has the values that need to be saved.

```
// No need to copy over, part of the starter code
@Composable
fun ItemEntryBody(
    //...
) {
    Column(
```

```

        // ...
    ) {
        ItemInputForm(
            itemDetails = itemUiState.itemDetails,
            //...
        )
        //...
    }
}

```

```

// Не нужно копировать, часть стартового кода
@Composable
fun ItemInputForm(
    itemDetails: ItemDetails,
    модификатор: Modifier = Modifier,
    onValueChange: (ItemUiState) -> Unit = {},
    enabled: Boolean = true
) {
    Column(modifier = modifier.fillMaxWidth(), verticalArrangement =
Arrangement.spacedBy(16.dp)) {
        OutlinedTextField(
            value = itemUiState.name,
            onValueChange = { onValueChange(itemDetails.copy(name = it)) },
            //...
        )
        OutlinedTextField(
            value = itemUiState.price,
            onValueChange = { onValueChange(itemDetails.copy(price = it)) },
            //...
        )
        OutlinedTextField(
            value = itemUiState.quantity,
            onValueChange = { onValueChange(itemDetails.copy(quantity = it)) },
            //...
        )
    }
}

```

Добавьте обработчик щелчка на кнопку Сохранить Чтобы связать все воедино, добавьте обработчик щелчка на кнопку Save. Внутри обработчика щелчка вы запускаете корутину и вызываете `saveItem()`, чтобы сохранить данные в базе данных Room.

В файле `ItemEntryScreen.kt`, внутри композиционной функции `ItemEntryScreen`, создайте `val` с именем `coroutineScope` с помощью композиционной функции `rememberCoroutineScope()`.

Примечание: `RememberCoroutineScope()` - это композитная функция, которая возвращает `CoroutineScope`, привязанный к композиции, в которой она была вызвана. Вы можете использовать композитную функцию `rememberCoroutineScope()`, когда хотите запустить корутину вне композита и убедиться, что корутина будет отменена после того, как область видимости

покинет композит. Вы можете использовать эту функцию, когда вам нужно управлять жизненным циклом корутин вручную, например, чтобы отменить анимацию при наступлении пользовательского события.

```
import androidx.compose.runtime.rememberCoroutineScope

val coroutineScope = rememberCoroutineScope()
```

Обновите вызов функции `ItemEntryBody()` и запустите корутину внутри лямбды `onSaveClick`.

```
ItemEntryBody()
    // ...
    onSaveClick = {
        coroutineScope.launch {
        }
    },
    modifier = modifier.padding(innerPadding)
)
```

Посмотрите на реализацию функции `saveItem()` в файле `ItemEntryViewModel.kt` для проверки валидности `itemUiState`, преобразования `itemUiState` в тип `Item` и вставки его в базу данных с помощью `itemsRepository.insertItem()`.

```
// Не нужно копировать, вы уже реализовали это как часть реализации Room

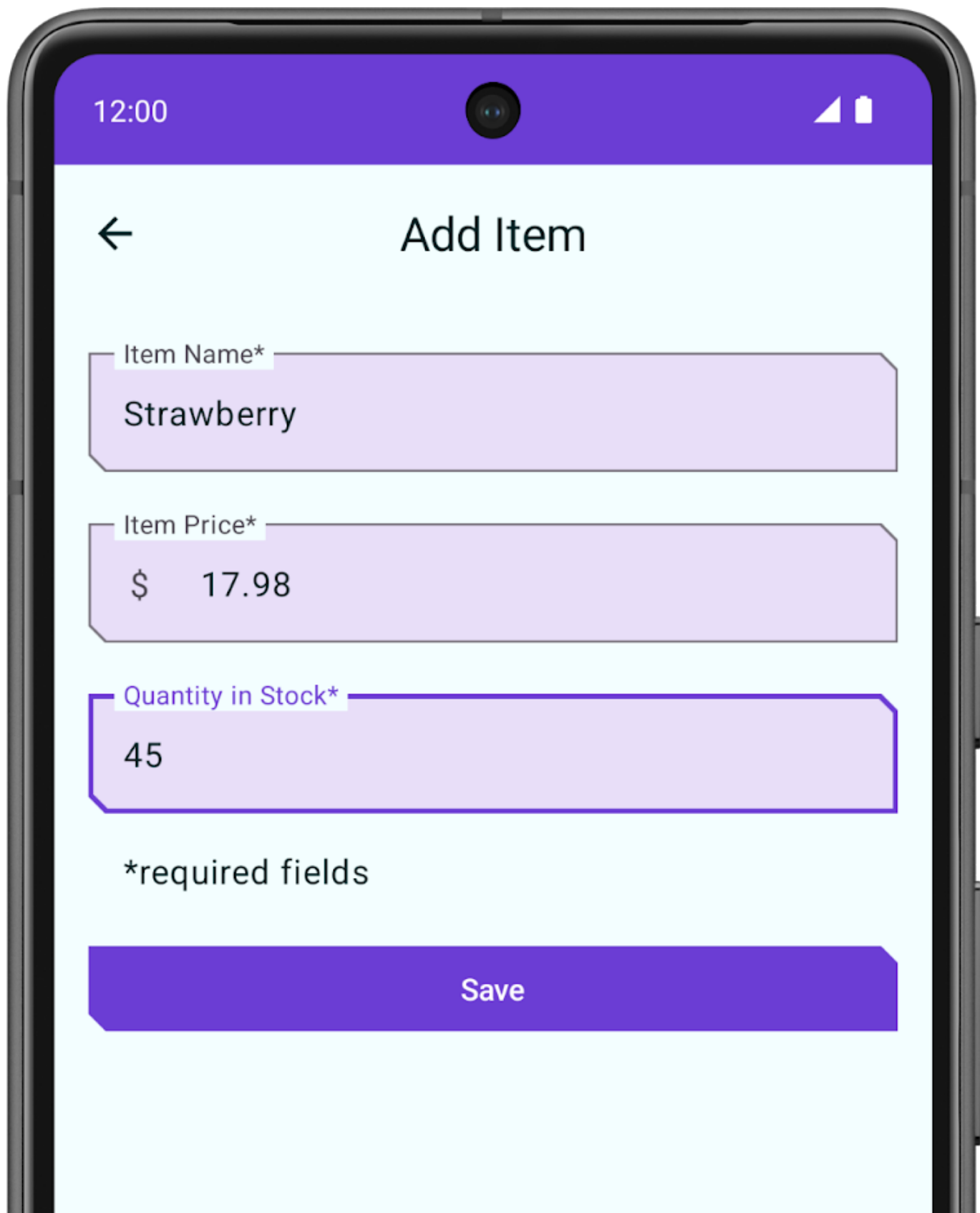
suspend fun saveItem() {
    if (validateInput()) {
        itemsRepository.insertItem(itemUiState.itemDetails.toItem())
    }
}
```

В `ItemEntryScreen.kt`, внутри композитной функции `ItemEntryScreen`, внутри корутины, вызовите `viewModel.saveItem()`, чтобы сохранить элемент в базе данных.

```
ItemEntryBody(
    // ...
    onSaveClick = {
        coroutineScope.launch {
            viewModel.saveItem()
        }
    },
    //...
)
```

Обратите внимание, что вы не использовали `viewModelScope.launch()` для `saveItem()` в файле `ItemEntryViewModel.kt`, но это необходимо для `ItemEntryBody()`, когда вы вызываете метод хранилища. Вы можете вызывать приостанавливающие функции только из корутины или другой приостанавливающей функции. Функция `viewModel.saveItem()` является приостанавливающей функцией.

Создайте и запустите свое приложение. Нажмите кнопку + FAB. На экране «Добавить элемент» добавьте сведения об элементе и нажмите «Сохранить». Обратите внимание, что нажатие кнопки Save не закрывает экран Add Item.



12:00

← Add Item

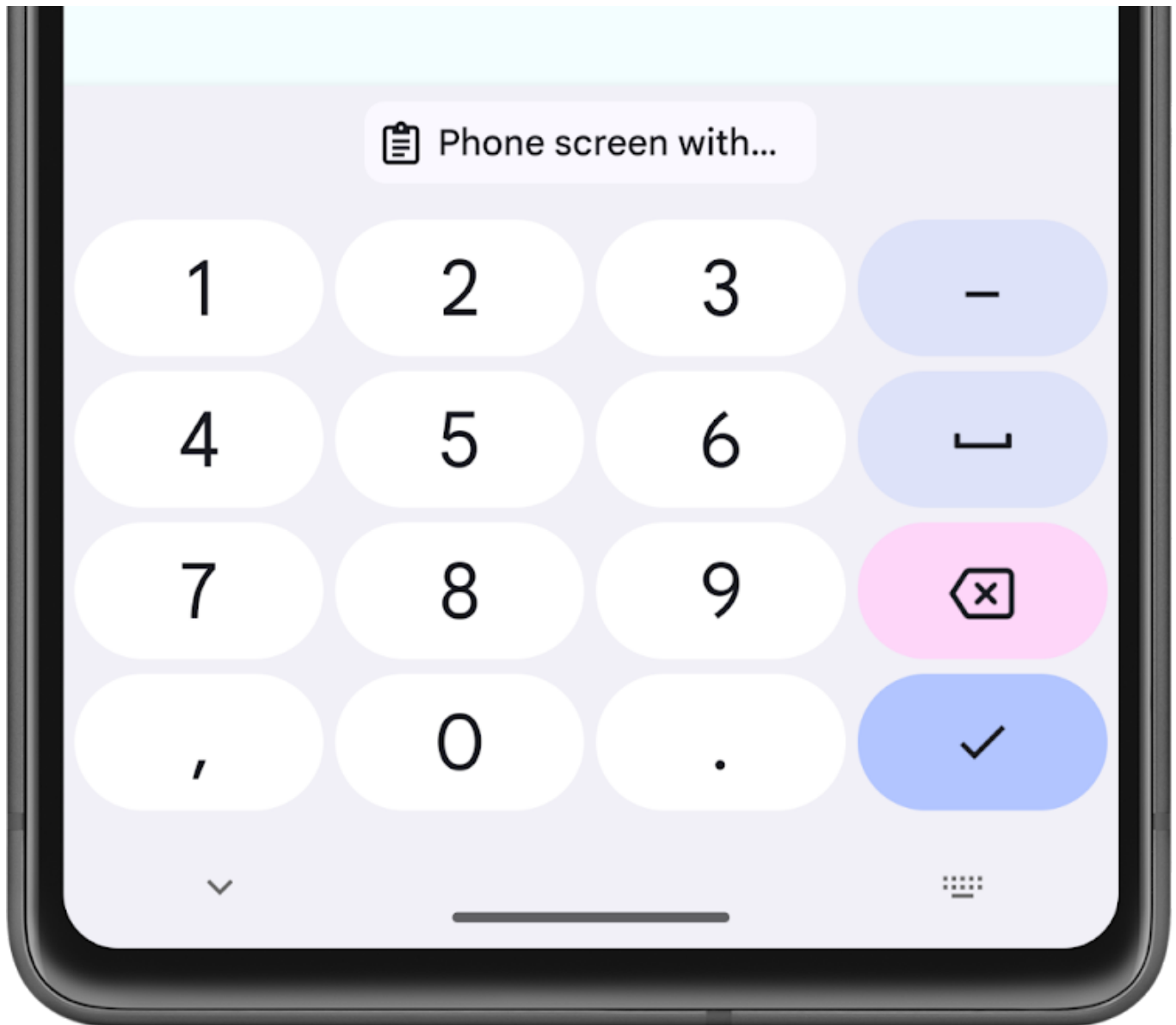
Item Name*
Strawberry

Item Price*
\$ 17.98

Quantity in Stock*
45

*required fields

Save



В лямбде `onSaveClick` добавьте вызов `navigateBack()` после вызова `viewModel.saveItem()` для перехода к предыдущему экрану. Ваша функция `ItemEntryBody()` выглядит следующим образом:

```
ItemEntryBody(
    itemUiState = viewModel.itemUiState,
    onItemValueChange = viewModel::updateUiState,
    onSaveClick = {
        coroutineScope.launch {
            viewModel.saveItem()
            navigateBack()
        }
    },
    modifier = modifier.padding(innerPadding)
)
```

Запустите приложение снова и выполните те же действия, чтобы ввести и сохранить данные. Обратите внимание, что на этот раз приложение переходит обратно на экран инвентаризации. Это действие

сохраняет данные, но вы не можете увидеть данные инвентаризации в приложении. В следующей задаче вы используете Инспектор базы данных для просмотра сохраненных данных.

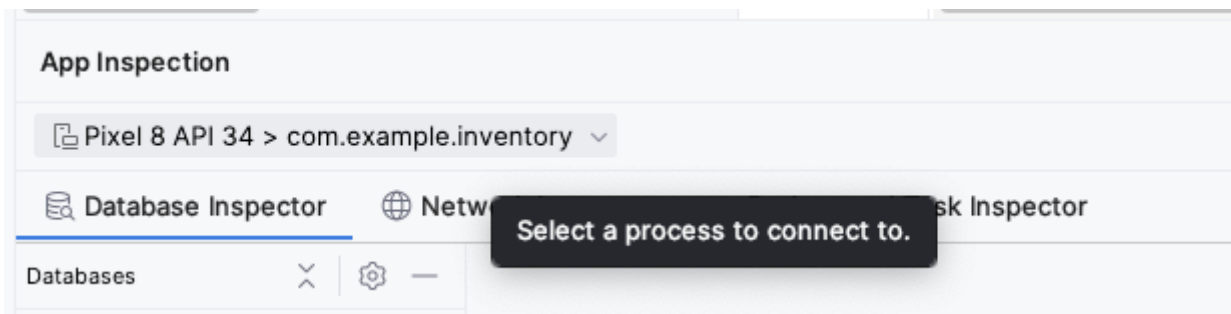


10. Просмотр содержимого базы данных с помощью инспектора базы данных

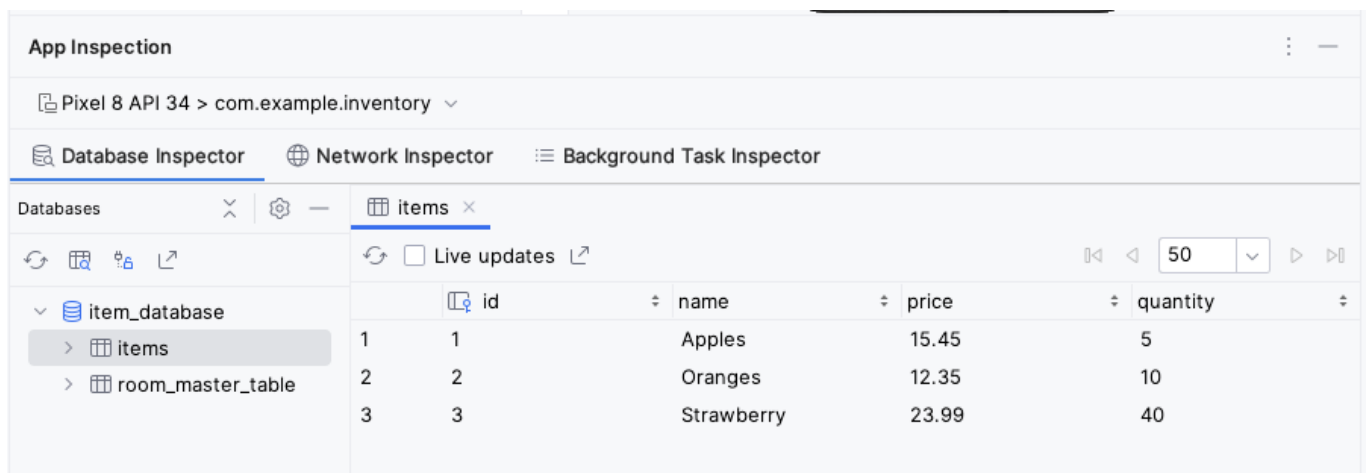
Инспектор баз данных позволяет просматривать, запрашивать и изменять базы данных вашего приложения во время его работы. Эта функция особенно полезна для отладки баз данных. Инспектор базы данных работает с обычным SQLite и с библиотеками, построенными поверх SQLite, такими как Room. Инспектор баз данных лучше всего работает на эмуляторах/устройствах с API уровня 26.

Примечание: Инспектор баз данных работает только с библиотекой SQLite, входящей в состав операционной системы Android на уровне API 26 и выше. Он не работает с другими библиотеками SQLite, которые вы поставляете в комплекте с вашим приложением.

Запустите приложение на эмуляторе или подключенном устройстве с уровнем API 26 или выше, если вы еще не сделали этого. В Android Studio в строке меню выберите View > Tool Windows > App Inspection. Выберите вкладку Инспектор базы данных. В панели Инспектора баз данных выберите базу данных com.example.inventory из выпадающего меню, если она еще не выбрана. База данных item_database в приложении Inventory появится на панели Databases.



Раскройте узел базы данных item_database на панели баз данных и выберите пункт Item to inspect. Если панель «Базы данных» пуста, с помощью эмулятора добавьте в базу данных несколько элементов с помощью экрана «Добавить элемент». Установите флажок Live updates в инспекторе базы данных, чтобы автоматически обновлять данные, которые он представляет, при взаимодействии с запущенным приложением в эмуляторе или на устройстве.



Поздравляем! Вы создали приложение, которое может сохранять данные с помощью Room. В следующем уроке вы добавите в приложение lazyColumn для отображения элементов в базе данных, а также добавите в приложение новые функции, например, возможность удалять и обновлять сущности. До встречи!

Получите код решения

Код решения для этого урока находится в репозитории GitHub. Чтобы загрузить код готового урока, воспользуйтесь следующими командами git:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git
```



```
$ cd basic-android-kotlin-compose-training-inventory-app  
$ git checkout room
```

Резюме

- Определите ваши таблицы как классы данных, аннотированные с помощью `@Entity`. Определите свойства, аннотированные с помощью `@ColumnInfo`, как столбцы в таблицах.
- Определите объект доступа к данным (DAO) как интерфейс, аннотированный `@Dao`. DAO сопоставляет функции Kotlin с запросами к базе данных.
- Используйте аннотации для определения функций `@Insert`, `@Delete` и `@Update`.
- Используйте аннотацию `@Query` со строкой запроса SQLite в качестве параметра для любых других запросов.
- Используйте Database Inspector для просмотра данных, сохраненных в базе данных Android SQLite.